COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

RECURRENT SELF-ORGANISING NETWORKS

Master thesis

2022 Bc. Ema Adamová

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

RECURRENT SELF-ORGANISING NETWORKS

Master thesis

Study program:	Applied informatics
Branch of study:	Applied informatics
Department:	Department of Applied Informatics
Supervisor:	prof. RNDr. Kristína Malinovská, PhD.
Consultant:	

Bratislava, 2025

Bc. Ema Adamová

35859437

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

PRIHLÁŠKA NA ZÁVEREČNÚ PRÁCU

Meno a priezvisko študenta: Bc. Ema Adamová
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: 18. - informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Recurrent self-organising networks

Anotácia: Self-organizing maps (SOMs) [1] are one of the classical neural network models that do not require a training signal and yet capture similarities in the training data and do so in a topological manner, that is, they capture similarities between data as distances in space. There are also recurrent versions such as MSOM [2] or RecSOM that are capable of capturing sequences of data in this way. The MSOMs are utilized, for example, in cognitive robotics. In classical SOMs, the values of the weights of the neurons in the map can be considered as models of the data that the network has learned. However, for recurrent SOMs, this relationship is not straightforward and reconstructing learned sequences is an interesting problem [2]. Another interesting phenomenon that has been addressed using self-organization is continuous learning and avoiding catastrophic forgetting [3]. The implementation and testing of the aforementioned algorithms related to recurrent self-organizing networks is very important for their use in future research.

Cieľ: The goal of the thesis is to implement recurrent SOM, namely MSOM and RecSOM in Python and test the implementation on suitable known test problems. Furthermore, the thesis shall investigate the possibility of reconstructing the stored sequences as well as the possibility of visualizing the information in the network and to implement such algorithms. The next task is to study and implement continuous learning for sequence self-organization and test the solution in a catastrophic interference problem. The final goal of this thesis is to integrate and test the proposed implementation in the context of ongoing research in cognitive robotics.

Literatúra:
[1] Kohonen, T., 1990. The self-organizing map. Proceedings of the IEEE, 78(9), pp.1464-1480.
[2] Strickert, M. and Hammer, B., 2005. Merge SOM for temporal data. Neurocomputing, 64, pp.39-71.
[3] Parisi, G.I., Tani, J., Weber, C. and Wermter, S., 2018. Lifelong learning of spatiotemporal representations with dual-memory recurrent self-organization. Frontiers in neurorobotics, 12, p.78.

Vedúci: RNDr. Kristína Malinovská, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky

35859437

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum schválenia: 09.12.2024

podpis študenta

...

. . . . . .

Bratislava, 2025 Bc. Ema Adamová

Acknowledgement

...

Abstract

Self-organizing maps (SOMs) are a classical model of unsupervised learning that can project high-dimensional data onto a low-dimensional grid while preserving topological structure. This makes them useful for visualization and clustering, but in their standard form they are limited to static datasets. Many real-world signals, however, are inherently temporal: they are best described as sequences, trajectories, or time series rather than isolated samples.

This thesis focuses on recurrent extensions of self-organizing maps that can process such temporal data. In particular, we study the Recurrent Self-Organizing Map (RecSOM) and the Merge Self-Organizing Map (MSOM). Both models introduce internal memory and context into the SOM framework, allowing the network to react differently to the same input depending on its recent history.

The work includes the implementation of SOM, RecSOM and MSOM in Python. The thesis further investigates how temporal context is represented inside these networks, how well they can organize and reconstruct sequential patterns, and how they can be applied in continual-learning scenarios with catastrophic forgetting, with an outlook towards applications in cognitive robotics.

Keywords: self-organizing map, recurrent self-organizing map, RecSOM, merge self-organizing map, MSOM, unsupervised learning, temporal sequences, continual learning, catastrophic forgetting, cognitive robotics

Abstrakt

Self-Organizing Maps (SOM) patria medzi klasické modely nesupervizovaného učenia, ktoré prevádzajú vysokodimenzionálne dáta na nízkodimenzionálnu mriežku pri zachovaní topologickej štruktúry. Vďaka tomu sú vhodné na vizualizáciu a klastrovanie, v základnej podobe však pracujú iba so statickými dátami. Mnohé reálne signály sú pritom prirodzene časové – opisujú sa skôr ako sekvencie, trajektórie alebo časové rady než ako izolované vzorky.

Táto práca sa zameriava na rekurentné rozšírenia Self-Organizing Maps, ktoré dokážu spracovávať sekvenčné dáta. Konkrétne skúmame modely Recurrent Self-Organizing Map (RecSOM) a Merge Self-Organizing Map (MSOM). Obe architektúry zavádzajú do SOM rámca vnútornú pamäť a kontext, takže sieť môže na rovnaký vstup reagovať odlišne v závislosti od svojej nedávnej histórie.

V práci implementujeme SOM, RecSOM a MSOM v jazyku Python. Ďalej skúmame, akým spôsobom tieto siete reprezentujú časový kontext, ako dobre dokážu organizovať a (prípadne) rekonštruovať sekvenčné vzory a ako sa správajú v scenároch continual learning s problémom catastrophic forgetting. Na záver naznačujeme možnosti využitia týchto modelov v oblasti kognitívnej robotiky.

Kľúčové slová: ...

Contents

1 Introduction **3**

2 Unsupervised Learning and Self-Organizing Networks **4**

2.1 Unsupervised learning 4

2.2 Self-Organizing Maps (SOMs) 5

2.2.1 Concept and motivation 5

2.2.2 Self-Organizing Map architecture 6

2.2.3 Best Matching Unit (BMU) 7

2.2.4 Neighborhood function and topology adaptation 8

2.2.5 Learning rule 8

2.2.6 Training process 9

2.2.7 Applications of Self-Organizing Maps 11

2.2.8 Limitations of Self-Organizing Maps 11

2.3 Recurrent Self-Organizing Maps (RSOMs) 12

2.3.1 Concept and motivation 12

2.3.2 RSOM architecture 13

2.3.3 Context trace and temporal processing 14

2.3.4 Learning rule 14

2.3.5 Training process 15

2.3.6 Applications of RSOMs 16

2.3.7 Limitations of RSOMs 17

2.4 Merge Self-Organizing Maps (MSOMs) 17

2.4.1 Concept and motivation 17

2.4.2 MSOM architecture 18

2.4.3 Context computation and temporal processing 18

2.4.4 Learning rule 19

2.4.5 Training process 20

2.4.6 Applications of MSOMs 20

2.4.7 Limitations of MSOMs 21

3 Research and Experimental Setup **22**
3.1 Datasets 22
3.1.1 Seeds dataset (static benchmark) 22
3.1.2 Mackey–Glass time series (temporal benchmark) 23
3.2 Preprocessing 23
3.3 Implementation Overview 23
3.3.1 SOM implementation 23
3.3.2 RSOM implementation 24
3.3.3 MSOM implementation 24
3.4 Hyperparameter Search Setup 24
3.4.1 SOM grid search 25
3.4.2 RSOM grid search 25

List of Figures

2.1 A conceptual overview of unsupervised learning (adapted from [2]). The model receives raw input data without labels and organizes it into meaningful structures based on similarity patterns. 4

2.2 Illustration of the Self-Organizing Map process. High-dimensional input data is projected onto a two-dimensional grid, preserving similarity relationships [4]. 6

2.3 Structure of a Self-Organizing Map. Each neuron in the two-dimensional grid is connected to all components of the input vector through its weight vector. 7

2.4 Examples of rectangular and hexagonal SOM grid layouts [4]. The chosen topology defines how neurons are arranged and how neighborhoods are formed. 7

2.5 The neighborhood radius decreases during training. Early updates affect many neurons; later updates focus on neurons close to the BMU. 8

2.6 Effect of a SOM weight update. The input vector x pulls the Best Matching Unit (BMU) and its neighboring neurons toward its position. 10

2.7 Schematic illustration of the RSOM architecture. The lower map represents the weight layer $\mathbf{w}$, which is connected to the input signal $s(t+1)$. The upper map represents the context layer $\mathbf{y}$. Its state $\mathbf{y}(t)$ is fed forward and combined with the new input to produce the next context state $\mathbf{y}(t+1)$, forming a recurrent loop [10]. 13

2.8 Illustration of the MSOM architecture. Each neuron has a weight vector $\mathbf{w}(t)$, a short-term pre-context $\mathbf{q}(t)$, and a long-term merged context $\mathbf{c}(t)$. The merged context determines BMU activation [8]. 19

List of Tables

Terminology

Terms

- **...**
  ...

Abbreviations

- **...** - ...

float

Motivation

...

Chapter 1

Introduction

Chapter 2

Unsupervised Learning and Self-Organizing Networks

2.1 Unsupervised learning

Unsupervised learning is a type of machine learning that finds patterns in data without labeled examples. In contrast to supervised learning, where each input is matched with a known output, unsupervised methods must figure out the structure in the data independently [5, 6]. This makes them especially helpful when there is a lot of raw data available, but it's not feasible to label it manually. In real-life situations, like analyzing sensor streams, exploring high-dimensional datasets, or identifying groups within structured data, unsupervised learning plays a key role.

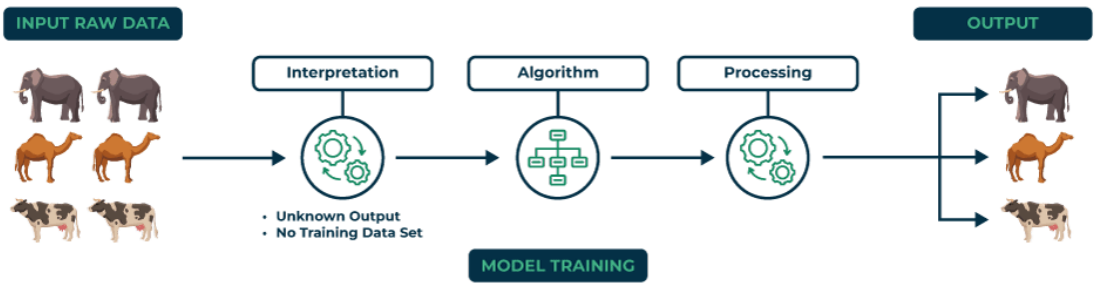

Figure 2.1: A conceptual overview of unsupervised learning (adapted from [2]). The model receives raw input data without labels and organizes it into meaningful structures based on similarity patterns.

The main objective of unsupervised learning is to find hidden structure within the data. This can mean clustering similar data points, reducing the dimensionality of datasets while preserving essential features, or identifying unusual patterns that deviate from what is considered normal [5, 3, 1]. Although these algorithms do not receive explicit feedback, they create internal representations of the data. These can

later be used for visualization, compression, detecting anomalies, or preparing data for more complex learning systems.

Two commonly used categories of unsupervised learning methods are *clustering* and *dimensionality reduction*. Clustering algorithms, such as *k-means* and hierarchical clustering, group data points based on their similarity [3]. Dimensionality reduction techniques, such as Principal Component Analysis (PCA) and t-SNE, transform high-dimensional data into lower-dimensional representations that that are still easy to understand [6]. These methods help uncover relationships or patterns that may not be obvious in the raw data.

Among various unsupervised learning methods, Self-Organizing Maps (SOMs) stand out for their ability to provide a structured and understandable representation of high-dimensional data. Unlike traditional clustering algorithms, SOMs arrange units on a two-dimensional grid. Similar inputs activate nearby neurons, forming a topologically meaningful map [4]. Because of this property, SOMs are often used in exploratory data analysis and visualization.

In the next sections, we will look closely at SOMs and also explore more advanced temporal versions, the Recurrent SOM (RSOM) [10] and the Multistream SOM (MSOM) [8].

2.2 Self-Organizing Maps (SOMs)

2.2.1 Concept and motivation

Self-Organizing Maps (SOMs) were introduced by Teuvo Kohonen in the 1980s. They were developed as a method for transforming high-dimensional data into a simpler, low-dimensional representation without supervision. Although many unsupervised learning techniques perform clustering or dimensionality reduction, they often lack an explicit notion of spatial structure. SOMs solve this problem by embedding data onto a two-dimensional grid in such a way that similar inputs are mapped to nearby locations, creating a smooth and easy to understand representation of complex datasets [4].

The main motivation for SOMs comes from the difficulty of understanding and visualizing high-dimensional data. In many fields such as neuroscience, robotics, finance, or environmental modeling, datasets often contain dozens or hundreds of features. Traditional clustering methods can find groups within these datasets, but they rarely show how these groups relate to each other. By preserving the topological relationships directly on a two-dimensional grid, SOMs make it possible to observe gradients, cluster boundaries, and transitions in the underlying data structure [4, 1, 3].

Another motivation comes from biological plausibility. SOMs are loosely inspired by patterns observed in the brain, such as sensory maps in the visual or auditory cor-

tex. Neurons responding to similar stimuli tend to be near one another. This biological organization appears in mechanisms such as competitive learning or neighborhood cooperation principles, which are mirrored in SOM training [4, 5]. When a neuron responds strongly to an input, not only that neuron but also its neighbors adjust their weights. This leads to smooth topographic regions that represent related inputs.

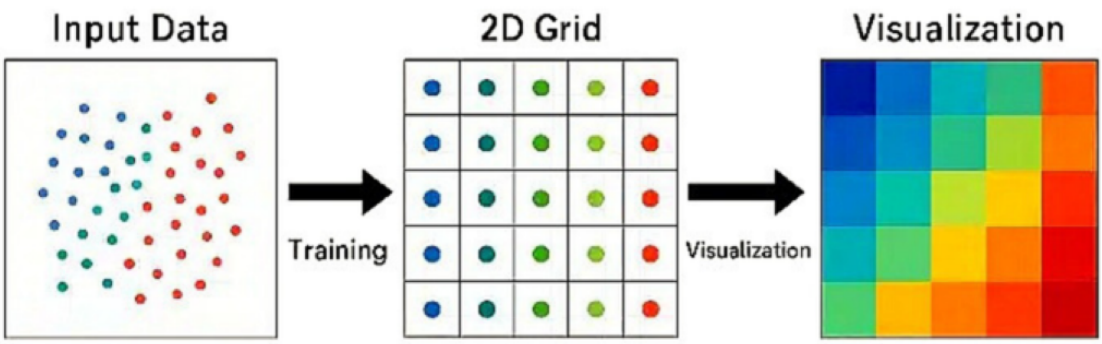

Figure 2.2: Illustration of the Self-Organizing Map process. High-dimensional input data is projected onto a two-dimensional grid, preserving similarity relationships [4].

Because of this, SOMs are especially useful for exploratory analysis. They help researchers find subtle relationships, visualize high-dimensional manifolds, and discover hidden structures that simpler clustering methods might miss [1].

2.2.2 Self-Organizing Map architecture

A Self-Organizing Map (SOM) is made up of a fixed grid of neurons arranged in a low-dimensional layout, usually a two-dimensional grid [4]. Each neuron i has an associated weight vector $\mathbf{w}_i \in \mathbb{R}^d$, where d is the number of features in the input data. These weight vectors are the parts of the network that get adjusted during training, and each one represents what kind of input patterns that neuron responds to.

The grid of neurons is often called the *feature map*. Even though this map is low-dimensional, each neuron represents a point in the original high-dimensional input space through its weight vector [4].

The topology of the grid defines how neurons are arranged and which neurons are considered neighbors. The most common layouts are *rectangular* and *hexagonal* grids [4]. Rectangular grids are easy to work with, while hexagonal grids give smoother transitions between neighboring neurons. Some SOMs also use *toroidal* boundaries, meaning the map wraps around at the edges, so the left and right sides (and top and bottom) are treated as neighbors. This helps avoid edge effects when the map is learning its structure [5].

Each neuron therefore has a fixed position on the grid, a weight vector in the input space, and a set of neighboring neurons determined by the chosen topology.

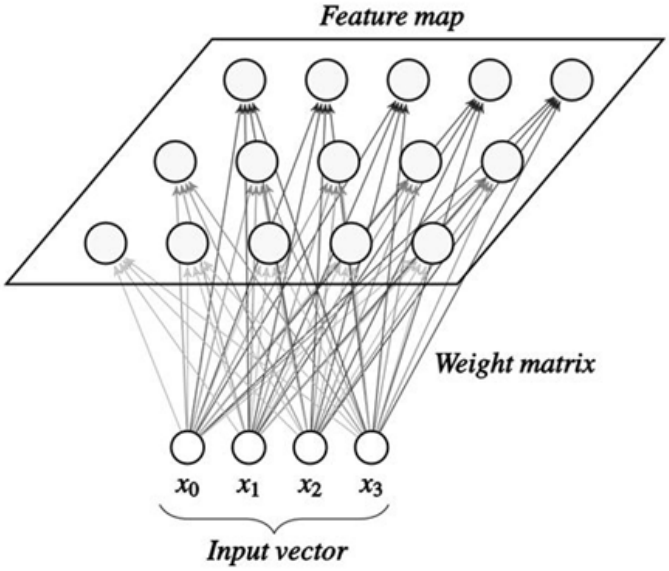

Figure 2.3: Structure of a Self-Organizing Map. Each neuron in the two-dimensional grid is connected to all components of the input vector through its weight vector.

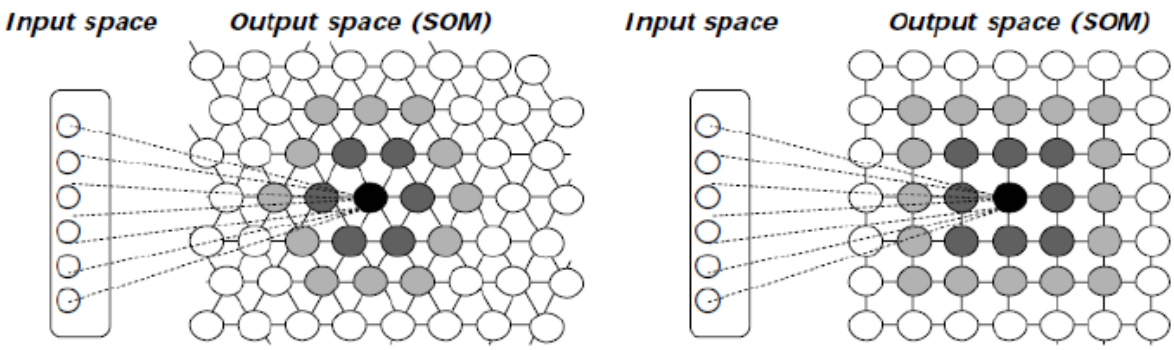

Figure 2.4: Examples of rectangular and hexagonal SOM grid layouts [4]. The chosen topology defines how neurons are arranged and how neighborhoods are formed.

2.2.3 Best Matching Unit (BMU)

When an input vector is presented to the SOM, the first step is to find the neuron whose weight vector is most similar to the input. This neuron is called the *Best Matching Unit* (BMU) [4]. The BMU marks the location on the map that best represents the given input pattern.

Formally, the BMU is defined as the neuron whose weight vector $\mathbf{w}_i$ is closest to the input vector $\mathbf{x}$ in terms of Euclidean distance:

$$i^* = \arg\min_i \|\mathbf{x} - \mathbf{w}_i\|.$$

The Euclidean distance can be written explicitly as:

$$\|\mathbf{x} - \mathbf{w}_i\| = \sqrt{(x_1 - w_{i1})^2 + (x_2 - w_{i2})^2 + \cdots + (x_d - w_{id})^2}.$$

This means that the SOM compares the input to all weight vectors and selects the neuron whose weight vector lies closest to the input in the high-dimensional input space. The BMU determines where on the map the influence of the input will be applied during learning [4].

2.2.4 Neighborhood function and topology adaptation

After the Best Matching Unit (BMU) is found, the SOM does not update only that neuron. Neurons that are close to the BMU on the grid should also be influenced. This idea is what allows the map to form smooth regions where neighboring neurons respond to similar inputs [4, 5].

The influence between a neuron i and the BMU i^* is described by the *neighborhood function*. A common choice is a Gaussian function:

$$h_{i,i^*}(t) = \exp\left(-\frac{\|r_i - r_{i^*}\|^2}{2\sigma(t)^2}\right),$$

where r_i and r_{i^*} are the grid positions of neuron i and the BMU, and $\sigma(t)$ is the neighborhood radius at time t.

At the start of training, $\sigma(t)$ is large, so many neurons around the BMU are updated together. As training continues, the radius becomes smaller, and only neurons very close to the BMU are affected [4]. This allows the map to learn broad structure first and refine smaller details later.

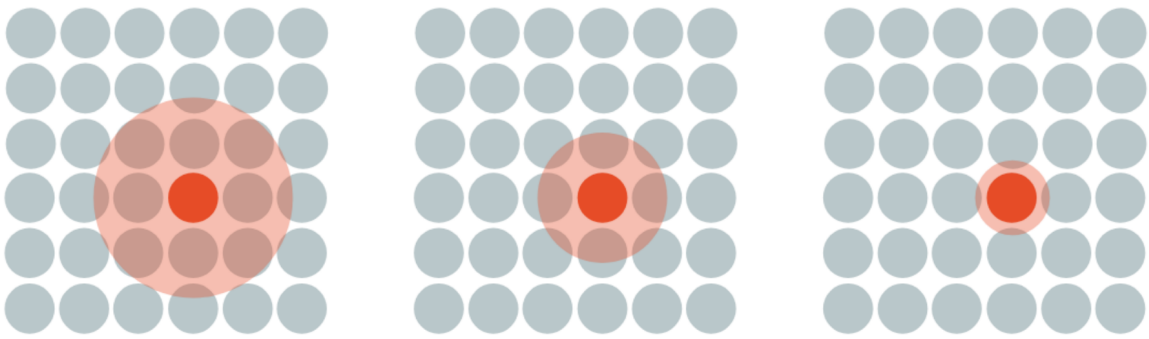

Figure 2.5: The neighborhood radius decreases during training. Early updates affect many neurons; later updates focus on neurons close to the BMU.

The neighborhood function is therefore important for preserving the topology of the map and for forming meaningful clusters on the grid [4].

2.2.5 Learning rule

After the neighborhood function is computed, the SOM updates the weight vectors of the BMU and its neighboring neurons. The update moves each weight vector slightly

toward the input vector. This is how the map gradually adapts to the structure of the data [4].

The update rule for a neuron i at time t is

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t)\, h_{i,i^*}(t)\, [\, \mathbf{x}(t) - \mathbf{w}_i(t)\,],$$

where

- $\alpha(t)$ is the learning rate,
- $h_{i,i^*}(t)$ is the neighborhood function,
- $\mathbf{x}(t)$ is the input vector,
- i^* is the BMU.

This rule has a simple interpretation: Each neuron moves its weight vector a little closer to the input, but the amount of movement depends on the learning rate and on how close the neuron is to the BMU. Neurons very close to the BMU receive a large update, while neurons far away receive almost no update [4].

2.2.6 Training process

Before the actual training begins, the SOM weight vectors must be initialized. A simple approach is *random initialization*, where each weight vector is assigned random values within the range of the input data. Another option is *sample initialization*, where some initial weight vectors are taken directly from real data points. Some implementations also use *PCA-based initialization*, which places the initial weights along the first principal components of the data. Although the map will reorganize itself during training, good initialization can help the SOM converge more smoothly and reduce early distortion [4, 1].

Once the initial weights are set, the training process can begin. Training a Self-Organizing Map takes place over many iterations, often grouped into epochs in which every input in the dataset is presented once. During each iteration, the map adjusts its weights slightly. Over time, these adjustments cause the neurons to organize into a meaningful structure that reflects the distribution of the data [4].

A single training iteration typically includes the following steps:

1. **Present an input vector.** At time step t, an input $\mathbf{x}(t)$ is taken from the dataset. In practice, the order of inputs is often randomized at the beginning of every epoch to avoid ordering effects [1].

2. **Find the BMU.** The Best Matching Unit i^* is identified as the neuron whose weight vector is closest to the input in Euclidean distance [4].

3. **Compute the neighborhood influence.** The neighborhood function $h_{i,i^*}(t)$ determines the strength of the update for each neuron. Neurons close to the BMU on the grid receive a larger influence, while distant neurons receive almost none [4, 5].

4. **Update the weights.** The BMU and the neurons in its neighborhood adjust their weight vectors toward the input according to the learning rule. This update slightly reshapes the map, preserving the relative relationships between neurons [4].

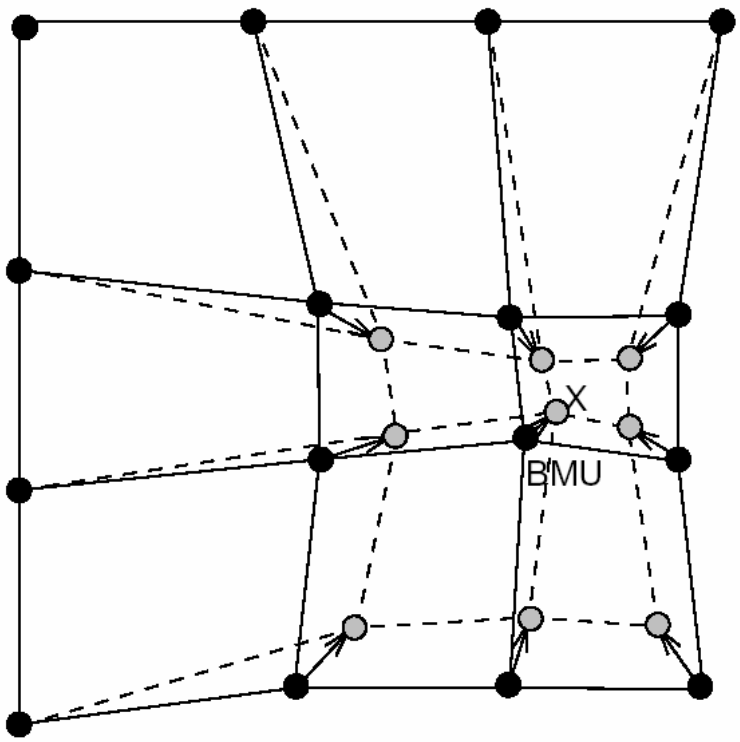

Figure 2.6: Effect of a SOM weight update. The input vector x pulls the Best Matching Unit (BMU) and its neighboring neurons toward its position.

5. **Decrease learning parameters.** Both the learning rate and the neighborhood radius decrease over time. A common way to define these decays is with exponential functions:

$$\alpha(t) = \alpha_0 \exp\left(-\frac{t}{\tau_\alpha}\right), \qquad \sigma(t) = \sigma_0 \exp\left(-\frac{t}{\tau_\sigma}\right),$$

where α_0 and σ_0 are the initial values, and τ_α, τ_σ control how quickly they decay. Large values at the start allow broad global adjustments. As training continues,

smaller values help refine the map without disrupting structures already learned [4].

These steps are repeated for every input vector and across many epochs. Large-scale organization is done during the early stages of training, while later epochs focus on fine-tuning the map. Eventually, the SOM reaches a stable configuration in which neighboring neurons respond to similar patterns in the input space [4, 1].

2.2.7 Applications of Self-Organizing Maps

Self-Organizing Maps have been applied in many practical settings where complex datasets need to be organized or compared in a structured way. The following examples illustrate how SOMs are used in different domains.

One common use is in *exploratory data analysis*. In areas such as finance or environmental modeling, SOMs help group similar observations and highlight relationships between different parts of the dataset. For example, a SOM trained on financial indicators can cluster companies or market regimes and provide a compact overview that supports further analysis [4].

SOMs are also frequently used in *biology and medicine*. In gene expression studies, they can organize genes or samples according to their activity patterns, revealing functional groups or shared biological responses. In medical signal analysis, such as EEG or ECG data, SOMs can group typical signal patterns and help identify segments that deviate from expected activity [1].

In *robotics and control*, SOMs are used to build internal representations of sensorimotor spaces. A robot can learn a map linking joint configurations or end-effector positions to locations on the SOM grid. Such representations support tasks like reaching, navigation, and adaptation to changes in the environment by providing a structured encoding of similar movements [4, 5].

Across these domains, SOMs are mainly used as tools for analyzing, organizing, or visualizing data rather than as predictive models. Their ability to produce a coherent and interpretable map makes them useful in situations where understanding the structure of the data is more important than making explicit predictions [3].

2.2.8 Limitations of Self-Organizing Maps

Although SOMs are effective tools for organizing and visualizing complex datasets, they also have limitations that restrict the range of problems they can address. One important limitation is that a standard SOM processes each input independently. The model has no internal memory, meaning that temporal or sequential patterns cannot be captured. This becomes problematic in applications such as speech processing,

movement analysis, or time-dependent sensor data, where the order of inputs carries essential information.

Another limitation comes from the fixed grid structure. The number of neurons and the overall shape of the map must be chosen in advance, and inappropriate choices can lead to maps that are either too coarse or unnecessarily large. Because the grid cannot grow or reorganize itself, the SOM may struggle if new patterns appear in the data that were not represented during training [1].

SOMs can also be sensitive to initialization and training parameters such as the learning rate, neighborhood radius, and number of iterations. Although the model typically converges to a meaningful structure, poor parameter choices may slow training or lead to distortions in some regions of the map [3].

These limitations do not reduce the usefulness of SOMs for static datasets, but they do highlight the need for models that can incorporate temporal context or multiple input streams. This motivates the development of extensions such as the Recurrent SOM (RSOM) [10] and the Multistream SOM (MSOM) [8], which introduce mechanisms for handling sequential information during learning.

2.3 Recurrent Self-Organizing Maps (RSOMs)

2.3.1 Concept and motivation

Standard Self-Organizing Maps process each input independently, without taking into account how earlier inputs may have influenced the system [4]. This works well for static datasets, but many real-world signals—such as speech, movement trajectories, or physiological recordings are inherently sequential. In such cases, the meaning of an input often depends on the sequence of inputs that came before it, not only by the current value.

The Recurrent Self-Organizing Map (RSOM), introduced by Voegtlin [10], extends the SOM framework by introducing a simple form of temporal memory. Instead of relying solely on the current input, the RSOM maintains a context trace for each neuron that captures recent input–weight differences through a decaying average. This means that the RSOM can respond differently to the same input depending on its temporal context.

By incorporating this temporal context, the RSOM becomes capable of organizing and clustering time-dependent patterns. Trajectories, rhythms, and other time-dependent structures that cannot be represented by a standard SOM can be mapped coherently while preserving the familiar two-dimensional grid [10].

2.3.2 RSOM architecture

The Recurrent Self-Organizing Map (RSOM) builds directly on the standard SOM architecture [4]. It keeps the same two-dimensional grid of neurons, the same topology, and the same weight vectors $\mathbf{w}_i$ that connect each neuron to the input space. As in the SOM, these weight vectors represent the prototypes that the neurons have learned for different regions of the input space.

The main structural addition in the RSOM is a separate *context layer*. This idea originates from the recurrent extension proposed by Voegtlin [10]. The context layer has the same grid structure as the main map and contains one context unit for each neuron. The context units are arranged in a one-to-one correspondence with the neurons in the weight layer. So every neuron has both a weight vector and an associated context state. While the weight vectors describe what each neuron represents, the context layer keeps track of recent activity of the map.

The context layer is connected recurrently over time. Its previous state $\mathbf{y}(t)$ influences how the system interprets the next input, effectively feeding the map's own recent history back into the learning process. This creates a temporal loop where the RSOM receives both the external input and its own internally generated context signal.

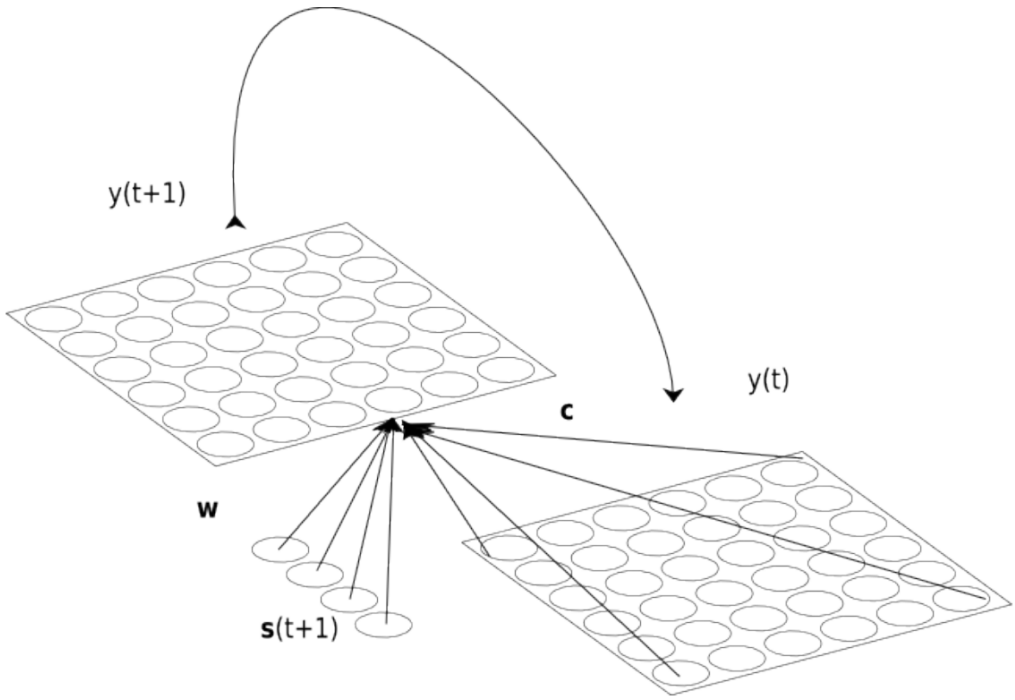

Figure 2.7: Schematic illustration of the RSOM architecture. The lower map represents the weight layer $\mathbf{w}$, which is connected to the input signal $s(t+1)$. The upper map represents the context layer $\mathbf{y}$. Its state $\mathbf{y}(t)$ is fed forward and combined with the new input to produce the next context state $\mathbf{y}(t+1)$, forming a recurrent loop [10].

In summary, the RSOM uses the same grid of neurons as a standard SOM but augments each neuron with an additional context unit and a recurrent connection over

time. This structural extension allows the network to incorporate temporal information into the learning process [10].

2.3.3 Context trace and temporal processing

The key mechanism of the Recurrent Self-Organizing Map lies in the way it incorporates temporal information. Instead of comparing the current input $\mathbf{x}(t)$ directly to each weight vector as in the standard SOM, the RSOM uses a *context trace* that summarizes recent input-weight differences [10]. This trace effectively stores a short-term history of how well each neuron matched past inputs.

For each neuron i, the context trace $\mathbf{c}_i(t)$ is computed as a decaying running average of past differences between the input and the neuron's weight vector:

$$\mathbf{c}_i(t) = (1-\beta)\,\mathbf{c}_i(t-1) + \beta\,\big(\mathbf{x}(t-1) - \mathbf{w}_i(t-1)\big),$$

where $\beta \in (0,1)$ determines how quickly older information is forgotten. Larger values of β emphasize recent inputs, while smaller values integrate information over a longer time span.

This context trace serves as a form of memory. Instead of comparing the current input directly with $\mathbf{w}_i$, the RSOM computes a *contextual distance* by comparing the input to the context trace:

$$D_i(t) = \|\mathbf{x}(t) - \mathbf{c}_i(t)\|.$$

The Best Matching Unit is then defined as

$$i^* = \arg\min_i D_i(t),$$

meaning the winning neuron is the one whose context trace best matches the current temporal pattern. Because the trace incorporates the recent input history, identical inputs may activate different neurons depending on their preceding sequence—something a standard SOM cannot achieve.

As a result, the RSOM naturally organizes temporal structures: inputs that tend to follow one another activate neighboring neurons, producing maps that reflect the dynamics of the underlying process.

2.3.4 Learning rule

The RSOM updates its weights in a way that is similar to the standard SOM, but with one important difference: the update is driven by the *context trace* rather than

directly by the input. This modification allows the learning process to reflect temporal dependencies in the data [10].

Once the Best Matching Unit i^* has been identified based on the contextual distance, the weights of the BMU and its neighbors are adapted. The update rule for neuron i at time t is

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t)\, h_{i,i^*}(t)\, \big[\, \mathbf{x}(t) - \mathbf{c}_i(t) \,\big],$$

where

- $\alpha(t)$ is the learning rate,
- $h_{i,i^*}(t)$ is the neighborhood function (as in the SOM),
- $\mathbf{c}_i(t)$ is the context trace of neuron i,
- i^* is the BMU at time t.

This rule differs from the SOM update in that it replaces the usual input–weight difference with the difference between the current input and the neuron's context trace. In practical terms:

- Neurons whose context matches the recent history of the input will receive stronger updates. - Neurons whose context does not align with the temporal structure of the input will adapt only weakly or not at all.

The neighborhood function is the same as in the standard SOM, meaning that nearby neurons on the grid update together. Thus, the RSOM retains the topology-preserving properties of the SOM while gaining sensitivity to temporal patterns.

As in the SOM, both the learning rate and the neighborhood radius typically decrease during training, enabling the network to first form a coarse temporal organization and later refine local structures.

2.3.5 Training process

Training a Recurrent Self-Organizing Map follows the same general structure as training a standard SOM, but with additional steps that update and use the context trace. These steps allow the network to incorporate temporal dependencies during learning [10]. Each training iteration operates on a single time step and gradually adjusts both the weight vectors and the internal context state of the map.

A typical training iteration consists of the following steps:

1. **Present an input vector.** At time step t, the current input $\mathbf{x}(t)$ is provided to the map. As with the SOM, the ordering of inputs is often randomized at the beginning of each epoch to avoid systematic biases.

2. **Update the context trace.** For each neuron i, the context trace $\mathbf{c}_i(t)$ is updated using a leaky integration of past input–weight differences:

$$\mathbf{c}_i(t) = (1-\beta)\,\mathbf{c}_i(t-1) + \beta\,\big(\mathbf{x}(t-1) - \mathbf{w}_i(t-1)\big).$$

   This step incorporates temporal information by maintaining a summary of recent mismatch between inputs and weights.

3. **Determine the BMU.** Instead of selecting the neuron whose weight vector is closest to the current input, the RSOM identifies the neuron whose *context trace* is closest:

$$i^* = \arg\min_i \|\mathbf{x}(t) - \mathbf{c}_i(t)\|.$$

   This ensures that the BMU depends not only on the present input but also on how the input sequence has evolved.

4. **Update the weights.** The BMU and its neighbors update their weight vectors according to the RSOM learning rule:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t)\,h_{i,i^*}(t)\,\left[\mathbf{x}(t) - \mathbf{c}_i(t)\right].$$

   As in the SOM, the neighborhood function $h_{i,i^*}(t)$ spreads the influence of the BMU over nearby neurons on the grid, allowing the map to maintain its topological organization.

5. **Decrease learning parameters.** The learning rate $\alpha(t)$ and neighborhood radius typically decrease gradually over time, following schedules similar to those used in the SOM. Early iterations focus on forming a coarse temporal organization of the map, while later iterations refine the learned structure without major reorganization.

These steps are repeated for every input in the sequence and across many epochs. Through the combination of context updates, temporal BMU selection, and weight adjustments, the RSOM learns to represent sequential patterns while preserving the SOM's spatial topology.

2.3.6 Applications of RSOMs

Thanks to its temporal memory, the RSOM can be applied to problems where the order of inputs is essential. It has been used in several domains, including:

- **Speech and signal processing.** RSOMs can cluster phoneme trajectories, EEG rhythms, or other time-varying signals by capturing how patterns evolve over time [10, 8].

- **Robotics and motor control.** Movement patterns and sensorimotor trajectories often carry important temporal dependencies. RSOMs have been used to organize these sequences and support tasks such as classification, prediction, or segmentation [7].

- **Time-series analysis.** RSOMs can detect dynamic regimes, cluster subsequences, and visualize system evolution, distinguishing states that look similar instantaneously but arise from different temporal contexts.

2.3.7 Limitations of RSOMs

While RSOMs improve upon SOMs by incorporating temporal context, they also have limitations that motivate the development of more flexible models such as the Multi-stream SOM (MSOM) [8].

First, each neuron maintains only a *single* context trace. This mixes short-term and long-term information and may not be sufficient to capture multi-scale temporal dynamics.

Second, the RSOM processes only one input stream. Many real-world systems involve multiple sensors or channels that evolve in parallel, and the RSOM cannot represent their interactions.

Third, the RSOM inherits structural constraints from the SOM, including the fixed grid size and sensitivity to hyperparameters such as the learning rate, neighborhood radius, and temporal decay parameter β.

Finally, because the BMU is chosen based solely on the context trace, the RSOM may struggle with noisy or abruptly changing sequences, where additional temporal mechanisms could be beneficial.

These limitations motivate the MSOM, which extends the RSOM by introducing multiple context channels and richer representations of temporal structure [8].

2.4 Merge Self-Organizing Maps (MSOMs)

2.4.1 Concept and motivation

The Recurrent Self-Organizing Map extends the SOM with a single temporal context trace, allowing it to process sequential data. However, many real-world temporal signals evolve on multiple timescales. Short, rapid changes and slower, longer-term trends

can both be important, but a single context trace—as used in the RSOM—mixes all temporal information together [10]. This limits its ability to distinguish between different temporal patterns.

The Merge Self-Organizing Map (MSOM), introduced by Strickert and Hammer [8], addresses this limitation by introducing two complementary temporal components: a short-term *pre-context* and a longer-term *merged context*. Instead of relying on a single memory signal, the MSOM uses these two traces to capture different temporal dynamics and merge them into a unified representation.

With this structure, the MSOM can cluster and organize temporal data that changes at different rates, while still preserving the familiar two-dimensional SOM grid.

2.4.2 MSOM architecture

The MSOM maintains the original SOM grid of neurons [4]. Each neuron i still has a weight vector $\mathbf{w}_i$ representing a prototype in the input space. The topology and neighborhood structure remain unchanged.

The key extension is that each neuron now has two additional context vectors:

- a *pre-context vector* $\mathbf{q}_i(t)$, which captures recent input–weight differences over a short timescale,
- a *merged context vector* $\mathbf{c}_i(t)$, which integrates information from the pre-context over a longer timescale.

Both context vectors follow the same grid layout as the weight layer, exactly as in the RSOM, but with an additional hierarchical structure: the pre-context feeds into the merged context, which then determines the BMU selection [8].

In summary, the MSOM augments each SOM neuron with two temporal memory traces, making it possible to separate short-term and long-term temporal dependencies.

2.4.3 Context computation and temporal processing

The MSOM computes its temporal context using two coupled update equations. The short-term *pre-context* is updated first:

$$\mathbf{q}_i(t) = \gamma\, \mathbf{q}_i(t-1) + (1-\gamma)\,\big(\mathbf{x}(t-1) - \mathbf{w}_i(t-1)\big),$$

where $\gamma \in (0,1)$ controls the decay rate. Small values emphasize recent inputs; larger values integrate information over a longer window.

Next, the long-term *merged context* is updated by integrating the pre-context:

$$\mathbf{c}_i(t) = \lambda\, \mathbf{c}_i(t-1) + (1-\lambda)\, \mathbf{q}_i(t),$$

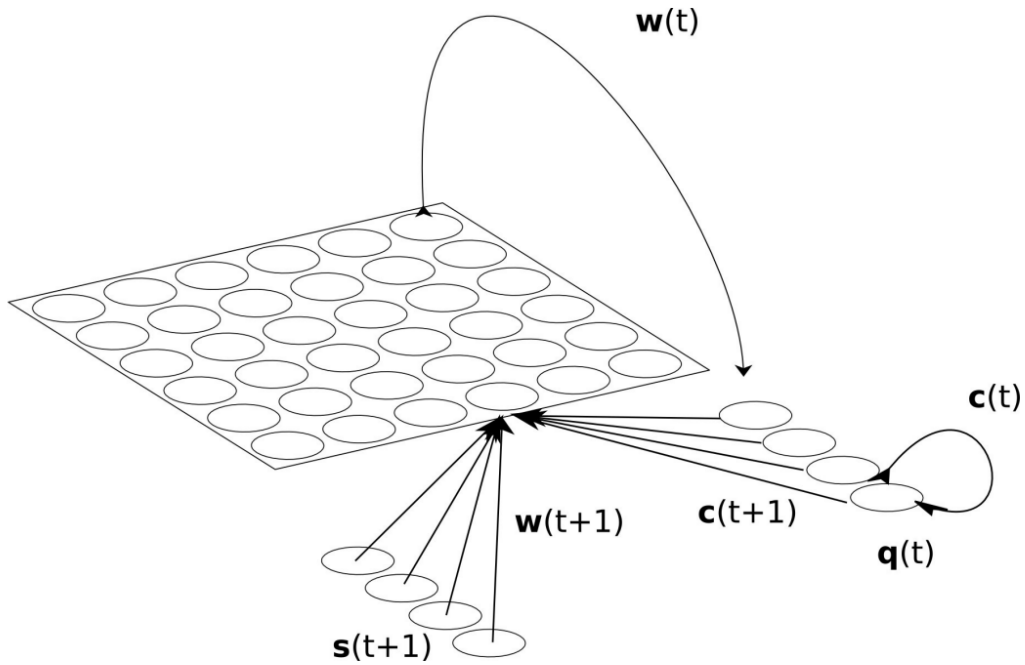

Figure 2.8: Illustration of the MSOM architecture. Each neuron has a weight vector $\mathbf{w}(t)$, a short-term pre-context $\mathbf{q}(t)$, and a long-term merged context $\mathbf{c}(t)$. The merged context determines BMU activation [8].

where $\lambda \in (0, 1)$ determines the timescale of the long-term memory [8].

The merged context is then used to determine the Best Matching Unit:

$$i^* = \arg\min_i \|\mathbf{x}(t) - \mathbf{c}_i(t)\|.$$

This parallels the RSOM, but the two-level context allows the MSOM to separate fast-changing temporal effects from long-term tendencies.

2.4.4 Learning rule

The MSOM uses a weight update rule similar to both the SOM and RSOM [4, 10], but the merged context replaces the single context trace:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t)\, h_{i,i^*}(t) \left[\mathbf{x}(t) - \mathbf{c}_i(t) \right].$$

Here,

- $\alpha(t)$ is the learning rate,
- $h_{i,i^*}(t)$ is the neighborhood function,
- $\mathbf{c}_i(t)$ is the merged context vector,

- i^* is the BMU chosen via contextual distance.

As in SOM and RSOM training, the learning rate and neighborhood radius typically shrink over time, allowing the network to move from coarse organization to fine temporal refinement.

2.4.5 Training process

A typical MSOM training iteration consists of:

1. **Present input.** The current input $\mathbf{x}(t)$ is provided to the model.
2. **Update the pre-context.** Short-term temporal differences are integrated into $\mathbf{q}_i(t)$.
3. **Update the merged context.** The longer-term context $\mathbf{c}_i(t)$ is updated from the pre-context.
4. **Select the BMU.** The BMU is chosen based on the merged context:

$$i^* = \arg\min_i \|\mathbf{x}(t) - \mathbf{c}_i(t)\|.$$

5. **Update the weights.** Weights are adapted according to the MSOM learning rule.
6. **Decrease learning parameters.** The learning rate and neighborhood radius decrease gradually, similar to SOM and RSOM training.

Across many epochs, the MSOM organizes sequential patterns while distinguishing between multiple temporal scales [8, 9].

2.4.6 Applications of MSOMs

The MSOM is useful in situations where temporal signals evolve on multiple timescales or where parallel streams interact. Applications include:

- **Multimodal robotics.** Motor trajectories, tactile signals, and proprioceptive cues can be integrated into a unified temporal map [7].
- **Physiological time-series.** EEG or EMG signals often have fast oscillations and slower trends; MSOMs can represent both.
- **Complex sequential structures.** Comparisons with other recursive SOM variants show that MSOMs can capture hierarchical or tree-like temporal patterns [9].

2.4.7 Limitations of MSOMs

Although more flexible than the RSOM, the MSOM still has limitations:

- Additional hyperparameters (γ, λ) increase tuning complexity.
- Very long-term dependencies may still be poorly preserved due to exponential decay.
- The fixed grid structure and schedule-based learning constraints of the SOM family remain [4].

Even with these limitations, the MSOM provides a practical way to model temporal patterns at multiple scales while preserving SOM topology.

Chapter 3

Research and Experimental Setup

This chapter covers the early stages of the experimental work done during the development of the recurrent self-organizing network models. At this point in the thesis, the main aim was not to achieve final results but to check that the implemented models acted reasonably and in line with their theoretical formulation. Since both RSOM and MSOM use temporal context mechanisms that can easily become unstable, a key part of the research involved ensuring the networks responded appropriately to input data and that the training dynamics matched expectations in existing literature.

To do this, I used several datasets for debugging and exploratory testing. I varied a number of hyperparameters to see how the models reacted. The following sections outline the datasets, implementation details, and the hyperparameter search approach.

3.1 Datasets

3.1.1 Seeds dataset (static benchmark)

To assess the accuracy of the classic SOM implementation, I used the well-known Seeds dataset from the UCI repository, included in the project as `seeds.txt`. The dataset includes seven numeric features that describe three types of wheat seeds. Although SOM does not utilize class labels during training, this dataset is commonly used to demonstrate cluster structures and validate SOM visualizations such as U-matrices or component planes.

In these experiments, this dataset served mainly as a sanity check. With its simple and well-known structure, it is easy to determine whether the SOM organizes the inputs meaningfully. For instance, if the component planes look noisy or the U-matrix does not show clear boundaries, it usually suggests an issue with weight initialization, learning rate scheduling, or neighborhood definition. Using this dataset made it simpler to verify that the basic SOM mechanics were implemented correctly.

3.1.2 Mackey–Glass time series (temporal benchmark)

To test the recurrent extensions of SOM, I used a segment of the Mackey–Glass chaotic time series, provided as `mackey_glass.xlsx`. This dataset is frequently used in the literature on time-series learning because it exhibits structured but unpredictable behavior. Small differences in the sequence can lead to different future states, which makes it ideal for evaluating temporal sensitivity.

In this thesis, the Mackey–Glass data was used primarily for debugging RSOM and MSOM. Both models should be able to distinguish different local temporal patterns, which can be observed directly in their BMU trajectories or in the evolution of the context vector. If the context vector grew unbounded, oscillated, or produced identical BMU patterns regardless of the input window, it indicated that something in the implementation was not working as intended.

Since this dataset is widely used in recurrent SOM literature, it also provided a useful reference: published RSOM and MSOM figures have characteristic shapes that allowed comparison with the behavior of my own implementations.

3.2 Preprocessing

Both datasets were preprocessed using an adaptive normalization method implemented in the function `smart_normalize` (in `utils.py`). This method chooses between StandardScaler, RobustScaler, or MinMax scaling based on the distribution of each feature. SOM-type models are sensitive to the scale of their input dimensions, and improper normalization can lead to degenerate maps. Automatic preprocessing ensured a stable input distribution and simplified debugging.

3.3 Implementation Overview

3.3.1 SOM implementation

The SOM baseline was implemented in a vectorized, GPU-compatible form (`SOM_vectorized.py`). The BMU search is performed efficiently using CuPy broadcasting. The implementation supports multiple configuration options:

- several weight initialization strategies (uniform, sampling, PCA, k-means),
- different grid distance metrics (Euclidean, Manhattan, Chebyshev, cosine, toroidal),
- various neighborhood kernels,
- customizable learning rate and neighborhood radius schedules.

This flexibility made it easier to explore different behaviors during the experimental phase. The model was evaluated using visualization tools such as the U-matrix, weight component plots, and BMU maps.

3.3.2 RSOM implementation

The RSOM implementation (`RSOM_cp_vectorized.py`) extends the SOM by introducing a recurrent context vector that evolves alongside the weight updates. RSOM compares the input not only to the neuron's weight vector but also to its context vector, weighted by parameters α and β.

Because the context update depends on previous time steps, RSOM is sensitive to parameter choices. Small mistakes in the implementation can lead to exploding context values or degenerate BMU sequences. For this reason, much of the early research focused on monitoring:

- the norm of the context vector over time,
- the smoothness of BMU trajectories,
- the magnitude of context adjustments per epoch.

These diagnostics were essential for verifying that RSOM behaved reasonably before moving on to more complex experiments.

3.3.3 MSOM implementation

The MSOM model (`MSOM.py`) uses a simpler temporal mechanism based on merging the current input with the previous BMU's state. Unlike RSOM, it does not maintain a full context matrix. Instead, each neuron stores two weight vectors: one for the input and one for the context. This structure typically produces more stable temporal dynamics while still enabling the model to differentiate between similar inputs appearing in different temporal situations.

MSOM therefore served as a useful reference model. When RSOM behaved unexpectedly, MSOM often produced smoother patterns, which helped identify whether the issue was architectural or implementation-related.

3.4 Hyperparameter Search Setup

To explore how different settings affect the learned maps, I designed an extensive hyperparameter search for all three models using parallel execution via `joblib`.

3.4.1 SOM grid search

The script `SOM_testing.py` performs a full-factorial search over:

- map dimensions (between 80 and 150 neurons total),
- initialization methods,
- grid distance metrics,
- neighborhood kernels,
- number of training epochs.

Each configuration is trained, and the final quantization error is recorded. Visualization tools were then used to verify that the best configuration produced an interpretable SOM.

3.4.2 RSOM grid search

The RSOM grid search (`RSOM_testing.py`) varied both standard SOM parameters and the temporal parameters α, β, γ_1, and γ_2. Because RSOM's behavior depends strongly on these values, systematic exploration was necessary to identify stable training regions.

Running many configurations in parallel made it easier to observe patterns, such as the fact that certain (β, γ_2) combinations consistently produced smoother BMU trajectories, while others led to unstable context updates.

Conclusion

Bibliography

[1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[2] GeeksforGeeks. Supervised vs. unsupervised learning. `https://www.geeksforgeeks.org/machine-learning/supervised-unsupervised-learning/`, 2024. Accessed: 2024-12-11.

[3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2009.

[4] Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.

[5] V. Kvasnička, L. Beňušková, J. Pospíchal, I. Farkaš, P. Tiňo, and A. Kráľ. *Úvod do teórie neurónových sietí*. Iris, Bratislava, 1997.

[6] Michael Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015.

[7] G. I. Parisi, J. Tani, C. Weber, and S. Wermter. Lifelong learning of spatiotemporal representations with dual-memory recurrent self-organization. *Frontiers in Neurorobotics*, 12:78, 2018.

[8] M. Strickert and B. Hammer. Merge som for temporal data. *Neurocomputing*, 64:39–71, 2005.

[9] M. Vančo and I. Farkaš. Experimental comparison of recursive self-organizing maps for processing tree-structured data. *Neural Networks*, 23(7):957–967, 2010.

[10] Thomas Voegtlin. A recurrent self-organizing map for temporal sequence processing. *Neural Networks*, 15(8-9):979–992, 2002.